AI-Powered Web Content Analyzer

This take-home assessment is designed to evaluate your full-stack development skills and practical understanding of AI integration. You will build and deploy a web application that accepts publicly accessible webpage URLs, retrieves and extracts the main content, and uses a Large Language Model (LLM) with thoughtfully designed prompts to generate structured, evidence-based analysis. The application should support content cleaning, validated JSON output, database storage, and a React interface where users can review the extracted source material, AI-generated results, prompt configurations, and previous analyses. We encourage thoughtful technical decisions, effective prompt engineering, and a reliable, user-friendly implementation.

1. Suggested Technology Stack

The following technologies are recommended. Candidates may select equivalent tools where permitted, but should document their choices.

Layer	Suggested technologies / requirements
Frontend	React (required); JavaScript or TypeScript; any UI library
Backend	Node.js + Express.js (required)
Database	Candidate's choice: PostgreSQL, MySQL, MongoDB, SQLite, or another suitable persistent database
Web extraction	Cheerio, Mozilla Readability, Playwright, or equivalent tools
LLM API	OpenAI, Anthropic, Gemini, or another suitable hosted LLM
Validation	Zod, JSON Schema, or equivalent schema validation
Deployment	Any suitable platform (e.g., Vercel, Render, Railway); no paid platform required

2. Functional Requirements

A. Web Content Extraction

- Accept a user-provided URL for a public HTML webpage. The application should not be hardcoded for one website.
- Fetch and extract the main readable content; reduce navigation, footer, scripts, and unrelated clutter.
- Show extracted source content in the interface.
- Handle invalid URLs, network failures, timeouts, and inaccessible pages with useful errors.
- Respect website access restrictions. Do not bypass authentication, CAPTCHA, or paywalls.

B. Prompt Engineering

- Design your own System Prompt and User Prompt.
- Provide two prompt versions (V1 and V2), and explain what changed and why.
- Explain how the prompts address hallucinations, missing information, and ambiguous source material.
- Test both versions and summarize differences in output quality.
- Include a Prompt Inspector in the UI showing prompt template/version, model choice, and output; never expose API keys.

C. LLM API Integration

- Call the model securely from the backend; never expose API credentials in browser code.
- Validate model output against a defined JSON schema before displaying/storing it.
- Handle provider errors and malformed/invalid outputs reasonably.
- Record model name, prompt version, URL, and timestamp for each analysis.

D. AI Analysis & Structured Output

Use an LLM to generate a validated JSON response containing the following:

```
{
  "title": "AI in Healthcare: Opportunities and Challenges",
  "summary": "This article explores how artificial intelligence is being applied in healthcare,
including clinical documentation, medical decision support, and administrative automation. It discusses
potential benefits such as improved efficiency and reduced administrative workload, while highlighting
challenges related to data privacy, accuracy, and system integration.",
  "keyFindings": [
    {
      "finding": "AI can help reduce administrative workload.",
      "evidence": "AI-powered tools can automate routine documentation tasks."
    },
    {
      "finding": "Data privacy remains a major concern.",
      "evidence": "Protecting sensitive patient information is essential when implementing AI systems."
    }
  ],
  "entities": [
    "Artificial Intelligence",
    "Healthcare",
    "Clinical Decision Support"
  ],
  "datesOrNumbers": [],
  "missingInformation": [
    "Specific cost savings from AI implementation",
    "Quantitative evidence of improvements in clinical outcomes"
  ]
}
```

Every key finding must be supported by evidence in the extracted webpage text. Unsupported information must not be presented.

E. Frontend & Database

- React UI must support URL input, extracted-text preview, structured analysis, Prompt Inspector, and analysis history.
- Provide clear loading, success, and error states.
- Persist analysis history in a database of your choice, including URL, analysis result, prompt version, model, and timestamp.
- History must remain available after the application server restarts.

3. Final Deliverables

Candidates must submit the following three deliverables:

Deliverable	Requirements
Live Deployment URL	• Provide a working, publicly accessible application URL. • Support URL input, content extraction, AI analysis, evidence display, Prompt Inspector, and saved history. • Securely configure LLM API credentials using server-side environment variables. • Ensure the application is fully functional during the technical review.

GitHub Repository Link	• Provide an accessible GitHub repository containing the complete source code. • Include frontend, backend, database integration, web scraping, and LLM integration logic. • Include Prompt V1/V2 templates, README.md with setup instructions, and .env.example. • Do not commit API keys or sensitive information. • Briefly disclose any AI-assisted coding tools used.
PDF Technical Report (3–5 Pages)	• Project Overview & Tech Stack: Frontend, backend, database, scraping approach, LLM/model, and deployment platform. • System Architecture: Implementation workflow, error handling, and JSON schema validation. • Prompt Engineering: System/User Prompt design, V1 vs V2 improvements, and hallucination-reduction strategies. • Application Screenshots: URL input/extraction, structured AI results, Prompt Inspector, and saved history. • Testing & Limitations: Results from three test webpages, known limitations, and suggested improvements.

4. Assessment & Technical Review

Reviewers may ask candidates to explain architecture decisions, the reason for their database and LLM choices, how evidence is verified, and how they would respond to invalid JSON, a long webpage, or prompt-injection text embedded in a webpage. A short live demonstration with an unseen URL may be included.

Bonus (optional): token/cost logging, streaming, background processing, evidence highlighting, automated regression tests, and prompt-injection defense. Bonus features do not replace required functionality.

Evaluation Rubric

Evaluation category	**Weight**
Prompt Engineering	30%
Backend & Web Scraping	25%
LLM Integration	20%
React Frontend	15%
Database Design & Code Quality	10%
TOTAL	**100%**

Evaluation prioritizes correct functionality, evidence-grounded AI results, maintainable engineering, and clear design reasoning over visual polish. Equivalent databases are welcome and are not scored by brand preference.